Assignment 1

Q1. **DAXPY Loop:** D stands for Double precision, A is a scalar value, X and Y are one-dimensional vectors of size 2^16 each, P stands for Plus. The operation to be completed in one iteration is X[i] = a*X[i] + Y[i]. Your task is to compare the speedup (in execution time) gained by increasing the number of threads. Start from a 2 thread implementation. How many threads give the max speedup? What happens if no. of threads are increased beyond this point? Why?

OUTPUT:

```
vprashar@DESKTOP-M9EFVQJ:~/UCS645/Assignments$ gcc A1Q1.c -fopenmp -o A1Q1
vprashar@DESKTOP-M9EFVQJ:~/UCS645/Assignments$ ./A1Q1
Threads:2       Time:0.000516   base_time:0.000516      speedup:1.000000
Threads:4       Time:0.000450   base_time:0.000516      speedup:1.144787
Threads:8       Time:0.011118   base_time:0.000516      speedup:0.046384
Threads:16      Time:0.001982   base_time:0.000516      speedup:0.260155
Threads:32      Time:0.003153   base_time:0.000516      speedup:0.163564
```

```
Performance counter stats for './A1Q1':

            49.06 msec task-clock                       #    1.137 CPUs utilized
              115      context-switches                 #    2.344 K/sec
               42      cpu-migrations                   #  856.126 /sec
              397      page-faults                      #    8.092 K/sec
   <not supported>      cycles
   <not supported>      instructions
   <not supported>      branches
   <not supported>      branch-misses

       0.043129380 seconds time elapsed

       0.014890000 seconds user
       0.041693000 seconds sys
```

Max Speedup of 1.14 was obtained using 4 threads.

Increasing the number of threads beyond 4 led to a rise in execution time and a drop in speedup, with performance becoming worse for 8, 16, and 32 threads.

This happens because the number of threads exceeds the available CPU cores, causing higher thread management overhead and frequent context switching.

Q2. **Matrix Multiply :** Build a parallel implementation of multiplication of large matrices (Eg. size could be 1000x1000). Repeat the experiment from the previous question for this implementation. Think about how to partition the work amongst the threads – which elements of the product array will be calculated by each thread? Implement 2 version of parallel implementation for given task. First, use 1D threading(i.e., make a single loop run in parallel). Second, use 2D threading (i.e., use nested looping to the most suitable loops to be parallelized)

OUTPUT:

```
vprashar@DESKTOP-M9EFVQJ:~/UCS645/Assignments$ gcc A1Q2.c -fopenmp -o A1Q2
vprashar@DESKTOP-M9EFVQJ:~/UCS645/Assignments$ ./A1Q2
1D Threading (Parallelizing outer loop only)
Threads:2       Time:1.889499   base_time:1.889499      speedup:1.000000
Threads:4       Time:1.623646   base_time:1.889499      speedup:1.163738
Threads:8       Time:1.055700   base_time:1.889499      speedup:1.789806
Threads:16      Time:0.915897   base_time:1.889499      speedup:2.063004
Threads:32      Time:0.876740   base_time:1.889499      speedup:2.155142

2D Threading (Parallelizing both i and j loops with collapse)
Threads:2       Time:1.762486   base_time:1.762486      speedup:1.000000
Threads:4       Time:1.542015   base_time:1.762486      speedup:1.142975
Threads:8       Time:1.062763   base_time:1.762486      speedup:1.658400
Threads:16      Time:0.891478   base_time:1.762486      speedup:1.977039
Threads:32      Time:0.891295   base_time:1.762486      speedup:1.977443

Direct Comparison: 1D vs 2D Threading
1D Threading Time: 1.063064 seconds
2D Threading Time: 1.099036 seconds
Speedup (1D vs 2D): 0.967269
```

```
Performance counter stats for './A1Q2':

        114296.44 msec task-clock                       #    6.324 CPUs utilized
            11746      context-switches                 #  102.768 /sec
              426      cpu-migrations                   #    3.727 /sec
             3144      page-faults                      #   27.507 /sec
  <not supported>      cycles
  <not supported>      instructions
  <not supported>      branches
  <not supported>      branch-misses

     18.072382469 seconds time elapsed

    114.132939000 seconds user
      0.175877000 seconds sys
```

The experiment implemented parallel matrix multiplication for 1000×1000 matrices using OpenMP with two parallelization strategies. The first approach used 1D threading to parallelize only the outermost loop, distributing complete rows among threads. The second approach used 2D threading with collapse(2) to parallelize both i and j loops simultaneously, distributing individual elements across threads.

Thread counts were varied from 2 to 32, and execution times and speedup were measured for each configuration. The results demonstrated that 1D threading offers better cache locality through row-wise distribution, while 2D threading provides finer-grained work distribution. The speedup measurements revealed the scalability characteristics and trade-offs between parallelization overhead and load balancing for each approach.

Q3. Calculation of π: This is the first example where many threads will cooperate to calculate a computational value. The task of this program will be arrive at an approximate value of π. The value of π is given by the following integral:

$$\pi = \int_0^1 \frac{4.0}{1+x^2} dx$$

This can be approximated as the sum of the areas of rectangles under the curve.

OUTPUT:

```
vprashar@DESKTOP-M9EFVQJ:~/UCS645/LAB1$ gcc A1Q3.c -fopenmp -o A1Q3
vprashar@DESKTOP-M9EFVQJ:~/UCS645/LAB1$ ./A1Q3
Number of steps: 100000

Parallel Computation with varying thread counts:
Threads:2       Time:0.000700   π = 3.141592653598146   speedup:1.000000
Threads:4       Time:0.000508   π = 3.141592653598127   speedup:1.378662
Threads:8       Time:0.008764   π = 3.141592653598125   speedup:0.079916
Threads:16      Time:0.003096   π = 3.141592653598125   speedup:0.226198
Threads:32      Time:0.004155   π = 3.141592653598127   speedup:0.168581
```

```
Performance counter stats for './A1Q3':

          138.32 msec task-clock                       #    3.742 CPUs utilized
             132      context-switches                 #  954.314 /sec
              51      cpu-migrations                   #  368.712 /sec
             145      page-faults                      #    1.048 K/sec
  <not supported>      cycles
  <not supported>      instructions
  <not supported>      branches
  <not supported>      branch-misses

      0.036966461 seconds time elapsed

      0.130522000 seconds user
      0.018003000 seconds sys
```

Numerical integration was used to calculate π by approximating $\int_0^1 4/(1+x^2)\,dx$ with 100,000 steps. The computation was parallelized using OpenMP with reduction(+:sum) to safely accumulate results across threads. Thread counts were varied from 2 to 32, and execution time and speedup were measured for each configuration. The results showed efficient scaling, with the reduction clause preventing race conditions, while the calculated value converged to the correct $\pi \approx 3.141592653589793$.

Rohan Bansal
102497012
3Q26